Kontextsensitive, lokale KI-Assistenz zur automatisierten Barrierefreiheitsanalyse von Rich-Client-Webanwendungen mittels Tool-Reports, Playwright-Interaktionen und Codeanalyse

Bachelorarbeit

vorgelegt am 11.05.2026

Fakultät Wirtschaft und Gesundheit

Studiengang Wirtschaftsinformatik

Kurs WWI2023

von

Joel Yerai Martinez Campo

Betreuung in der Ausbildungsstätte:

BITBW
Ilko Hoffman
Betreuer der Bachelorarbeit

Unterschrift

DHBW Stuttgart:

Sascha Alexander Ströbel

LaTeX-Vorlage für Projekt-, Seminar- und Bachelorarbeiten

Dies wird meine Bachelorarbeit

Inhaltsverzeichnis

Abkürzungsverzeichnis **V**

Abbildungsverzeichnis **VI**

Tabellenverzeichnis **VII**

1 Einleitung **1**
1.1 Motivation und Kontext . . . 1
1.2 Problemstellung . . . 3
1.3 Zielsetzung und Forschungsfragen . . . 4
1.4 Aufbau der Arbeit . . . 5

2 Theoretische Grundlagen **6**
2.1 Barrierefreiheit von Webanwendungen . . . 6
2.1.1 Begriffsdefinition und Relevanz . . . 6
2.1.2 Web Content Accessibility Guidelines (WCAG): Aufbau, Prinzipien und Konformitätsstufen . . . 8
2.1.3 Rechtlicher Rahmen . . . 9
2.2 Automatisierte Barrierefreiheitsanalyse . . . 10
2.2.1 Klassische Tool-Ansätze . . . 10
2.2.2 Grenzen bestehender Tools . . . 11
2.2.3 Taxonomie der Verstöße: Syntaktisch, Semantisch, Layout . . . 13
2.3 Rich-Client-Webanwendungen und Testautomatisierung . . . 13
2.3.1 Charakteristika und Abgrenzung . . . 13
2.3.2 Herausforderungen für die Barrierefreiheitsprüfung . . . 14
2.3.3 Playwright als Testautomatisierungswerkzeug . . . 15
2.4 KI-Assistenzsysteme für Softwareanalyse und Handlungsempfehlungen . . . 16

3 Methodik **17**
3.1 Wissenschaftliche Methode Fallstudienanalyse . . . 17
3.2 Untersuchungsgegenstand und Scope-Abgrenzung . . . 17
3.3 Evaluationsdesign und Kriterien . . . 17

4 Analyse und Anforderungen **18**
4.1 Analyse der Tool-Outputs und typischer Findings . . . 18
4.2 Analyse von Interaktionsdaten aus Playwright . . . 18
4.3 Analyse der Codebasis als Kontextquelle . . . 18
4.4 Datenschutz & Sicherheitsanforderungen . . . 18

5 Konzeption des Assistenzsystems **19**
5.1 Zielarchitektur und Datenpipeline . . . 19
5.2 Modell der Kontextualisierung & Priorisierung . . . 19
5.3 Ausgabeformat entwicklernahe To-Do-Liste . . . 19

6 Implementierung des Proof of Concept **20**
6.1 Technische Umsetzung . . . 20

6.2 Beispielabläufe und Ergebnisse am Untersuchungsobjekt 20

7 Evaluation & Diskussion **21**
7.1 Ergebnisse entlang der Kriterien und Limitationen 21
7.2 Handlungsempfehlungen und Ausblick 21

8 Fazit **22**

Anhang **23**

Literaturverzeichnis **26**

Abkürzungsverzeichnis

BFSG Barrierefreiheitsstärkungsgesetz

BITBW IT Baden-Württemberg

BITV Barrierefreie-Informationstechnik-Verordnung

DOM Document Object Model

DSGVO Datenschutz-Grundverordnung

EAA European Accessibility Act

HTML Hypertext Markup Language

LLM Large Language Model

TF Teilfragen

WCAG Web Content Accessibility Guidelines

WAI Web Accessibility Initivative

WHO World Health Organization

Abbildungsverzeichnis

1 Framework Popularity über die letzten Jahre[1] 14

[1] Vgl. Krotoff 2026

Tabellenverzeichnis

1 Vergleich der Erkennungsleistung verschiedener Accessibility-Prüfwerkzeuge (basierend auf Kodithuwakku und Wickramaarachchi). CER beschreibt den Anteil korrekt erkannter Verstöße im Verhältnis zu allen gemeldeten Verstößen, während ICR den Anteil tatsächlich vorhandener Accessibility-Probleme misst, die vom jeweiligen Tool erkannt wurden. 12
2 Konzeptmatrix (Hauptmatrix, hoch priorisierte Studien) 17
3 Konzeptmatrix (vollständige Übersicht) 25

1 Einleitung

Digitale Anwendungen sind für Verwaltung, Wirtschaft und Gesellschaft zu einer zentralen Infrastruktur geworden. Damit alle Menschen, unabhängig von individuellen Einschränkungen, gleichberechtigten Zugang zu Informationen und Dienstleistungen erhalten, gewinnt Web-Barrierefreiheit zunehmend an Bedeutung. In der Praxis zeigt sich jedoch eine Diskrepanz zwischen normativen Anforderungen (z. B. WCAG-basierte Kriterien)[2] und der tatsächlichen Umsetzung in Softwareprojekten: Barrierefreiheit wird häufig spät berücksichtigt, Prüfungen erfolgen punktuell und Korrekturen sind zeitaufwendig. Besonders in gewachsenen Systemlandschaften mit Legacy-Anteilen und knappen Entwicklungsressourcen entstehen so wiederkehrende Qualitätsdefizite.

Parallel dazu haben sich Large Language Model (LLM) als leistungsfähige Werkzeuge zur Unterstützung von Softwareentwicklung etabliert. Sie helfen, Aufgaben wie Code-Analyse, Refactoring oder die Erstellung technischer Empfehlungen zu beschleunigen. Für die Barrierefreiheit ist dies besonders relevant: Hier müssen nicht nur Fehler erkannt, sondern auch korrekt, standardkonform und kontextsensitiv behoben werden. Gleichzeitig bestehen in vielen Organisationen, insbesondere in sensiblen Umgebungen, strikte Vorgaben an Datenschutz und Informationssicherheit, wodurch Cloud-basierte Assistenzsysteme häufig nicht eingesetzt werden dürfen. Daraus ergibt sich die zentrale Herausforderung, LLM-Unterstützung so zu gestalten, dass sie lokal, nachvollziehbar und qualitätsgesichert in bestehende Entwicklungsprozesse integriert werden kann.

Diese Arbeit untersucht daher, wie ein LLM- bzw. agentenbasierter Ansatz zur automatisierten Erkennung und Behebung von Barrierefreiheitsproblemen in Webanwendungen konzipiert und prototypisch umgesetzt werden kann.[3] Im Fokus steht ein Workflow, der automatisierte Tests (z. B. via Browser-Automation und Accessibility-Linter) mit einer LLM-gestützten Analyse und Änderungsvorschlägen verbindet und die Änderungen anschließend erneut testet. Ziel ist es, den manuellen Aufwand zu reduzieren, die Konsistenz der Ergebnisse zu erhöhen und Barrierefreiheit als kontinuierlichen Bestandteil der Softwarequalität zu verankern.

1.1 Motivation und Kontext

Rund 1,3 Milliarden Menschen (16% der Weltbevölkerung) leben mit einer Form von Behinderung.[4] Für diese Personengruppe ist der Zugang zu digitalen Angeboten keine bloße Komfortfrage, sondern eine Voraussetzung für gesellschaftliche Teilhabe in nahezu allen Lebensbereichen: von der Kommunikation mit Behörden über den Zugang zu Bildung und Gesundheitsinformationen bis hin zur Teilnahme am wirtschaftlichen Leben. Das Recht auf barrierefreien Zugang zu Informations- und Kommunikationstechnologien ist völkerrechtlich in der UN-

[2] Vgl. World Wide Web Consortium (W3C) 2023
[3] Vgl. Bassi et al. 2025, S.297 ff.
[4] Vgl. World Health Organization 2026

Behindertenrechtskonvention verankert und hat sich in den vergangenen Jahren zunehmend auch in verbindlichem nationalen und europäischen Recht niedergeschlagen.[5]

Auf europäischer Ebene verpflichtet der European Accessibility Act (EAA) die Mitgliedstaaten, Barrierefreiheitsanforderungen für digitale Produkte und Dienstleistungen durchzusetzen.[6] In Deutschland wurde diese Richtlinie durch das Barrierefreiheitsstärkungsgesetz (BFSG) in nationales Recht überführt, das seit dem 28. Juni 2025 auch für weite Teile der Privatwirtschaft gilt.[7] Ergänzt wird dieser Rahmen durch die Barrierefreie-Informationstechnik-Verordnung (BITV), die für Behörden und öffentliche Stellen bereits seit Jahren verbindliche Vorgaben formuliert.[8][9]

Trotz dieser eindeutigen Rechtslage ist die praktische Umsetzung barrierefreier Webanwendungen nach wie vor unzureichend. Die jährlich durchgeführte WebAIM-Million-Studie, die die Startseiten der meistbesuchten Webseiten weltweit analysiert, stellte 2025 fest, dass 94,8% der untersuchten Seiten nachweisbare Verstöße gegen die WCAG aufweisen.[10] Damit bleibt Barrierefreiheit in der Entwicklungspraxis trotz gesetzlicher Verpflichtung systematisch unterrepräsentiert. Im Vergleich mit den letzten Jahren haben wir seit 2019 (97,8%) lediglich eine Verbesserung um 3,1 Prozentpunkte.[11]

Die IT Baden-Württemberg (BITBW) nimmt als zentraler IT-Dienstleister der Landesverwaltung Baden-Württemberg in diesem Kontext eine besondere Stellung ein. Am 1. Juli 2015 auf Grundlage des gleichnamigen BITBW-Gesetzes gegründet, löste sie das ehemalige Informatikzentrum Landesverwaltung ab und fungiert seither als dem Innenministerium untergeordnete Landesoberbehörde.[12] Zu ihren Kunden zählen Ministerien, Polizeibehörden, Kommunen sowie Stiftungen des Öffentlichen Rechts; ihre Aufgaben umfassen die „Bereitstellung, Betrieb und Ausbau der zentralen informationstechnischen Infrastruktur für die Landesverwaltung [...]", die „Sicherstellung der Informationssicherheit [...]“ sowie die Beschaffung von IT-Produkten und Lizenzen.[13]

Als Entwicklerin und Betreiberin öffentlicher Webanwendungen ist sie dabei gesetzlich zur Einhaltung der BITV und dem BFSG verpflichtet und trägt unmittelbare Verantwortung dafür, dass ihre Systeme für alle Bürgerinnen und Bürger zugänglich sind, unabhängig den jeweiligen Einschränkungen die diese Bürger haben könnten. Sie bildet damit den praktischen Rahmen der vorliegenden Arbeit.

[5] Vgl. United Nations Human Rights 2026
[6] Vgl. European Comission 2026
[7] Vgl. Barrierefreiheitsstärkungsgesetz 2026
[8] Vgl. Heinemann 2024, S. 281
[9] Vgl. Bundesministerium für Digitales und Staatsmodernisierung 2025
[10] Vgl. WebAIM 2025a
[11] Vgl. WebAIM 2019
[12] Vgl. BITBW 2021
[13] Landesrecht BW 2015

1.2 Problemstellung

Die mangelhafte Umsetzung von Barrierefreiheit in Softwareprojekten lässt sich nicht allein auf fehlende Motivation oder Ressourcen zurückführen. Ein wesentlicher Treiber sind die Schwächen der verfügbaren Werkzeuge und Tools: Automatisierte Barrierefreiheitsanalyse-Tools wie axe[14], WAVE[15] oder Lighthouse[16] erkennen zwar syntaktische Verstöße, stoßen jedoch bei semantischen Problemen an ihre Grenzen. Ob ein vorhandener Alternativtext den Bildinhalt tatsächlich angemessen beschreibt, oder ob eine Schaltflächenbeschriftung für Screenreader-Nutzerinnen und -Nutzer verständlich ist, entzieht sich der regelbasierten Prüflogik dieser Werkzeuge.

Empirisch belegt wird diese Schwäche durch das Mutation-Testing-Framework **Ma11y**: In einer systematischen Evaluation mit 25 gezielt eingebrachten Barrierefreiheits-Defekten erkannte kein einzelnes der sechs untersuchten Tools mehr als 52% der injizierten Fehler. Somit blieben im Durchschnitt nahezu die Hälfte aller Defekte unentdeckt.[17] Besonders aufschlussreich ist dabei die Differenzierung nach Verstoßtypen: Während syntaktische Defekte zu 54 Prozent erkannt wurden, lag die Erkennungsrate bei semantischen Verstößen lediglich bei 26 Prozent.[18] Sechs der 25 Defekttypen - darunter die Manipulation von Seitentiteln und Alternativtexten - wurden von keinem der untersuchten Tools erkannt.

Eine zweite strukturelle Schwäche betrifft die Charakteristik moderner Webanwendungen. Rich-Client-Anwendungen auf Basis von Frameworks wie React erzeugen ihren Inhalt größtenteils clientseitig und verändern den Document Object Model (DOM) dynamisch in Abhängigkeit von Nutzerinteraktionen. Barrieren entstehen häufig erst in bestimmten Zuständen: wenn ein Modalfenster geöffnet wird, eine Fehlermeldung erscheint oder ein Dropdown-Menü aktiviert ist. Rein statische Analysen - oder Analysen, die ausschließlich den initialen Seitenladezustand prüfen - erfassen diese zustandsabhängigen Barrieren nicht.[19]

Jüngere Forschungsarbeiten zeigen, dass LLMs geeignet sind, die Grenzen regelbasierter Tools zu überwinden. So erreicht das System **AccessGuru**[20] durch den Einsatz von GPT-4o eine Reduktion des Violation Scores um bis zu 84% , und **GenA11y**[21] erzielt bei der Erkennung von Barrierefreiheitsproblemen eine Precision von 94,5%. Diese Ansätze beschränken sich jedoch auf statisches Hypertext Markup Language (HTML). Keiner der bekannten Ansätze verknüpft Tool-Reports, Playwright-Interaktionsdaten und den Quellcode der geprüften Komponenten zu einem integrierten Analysesystem.[22] Der Schritt von der Fehlererkennung zu einer entwicklernahen, direkt umsetzbaren Handlungsempfehlung - idealerweise mit Bezug zur konkreten Quellcodestelle - wird in der Literatur zwar als offenes Problem benannt,[23] bleibt aber bislang ungelöst.

[14] Vgl. Deque Systems 2026
[15] Vgl. WebAIM 2025b
[16] Vgl. Google 2025
[17] Vgl. Tafreshipour et al. 2024, S. 1-12
[18] Vgl. Tafreshipour et al. 2024, S. 9
[19] Vgl. Bassi et al. 2025, S. 1-10
[20] Vgl. Fathallah, Hernández, Staab 2025
[21] Vgl. He, Huq, Malek 2025
[22] Vgl. Fathallah, Hernández, Staab 2025, S. 18
[23] Vgl. He, Huq, Malek 2025, S. 7 f.

Daraus ergibt sich eine klar umrissene Lücke: Es fehlt ein Assistenzsystem, das (a) die dynamischen Zustände einer React-Anwendung durch automatisierte Browser-Interaktionen erfasst, (b) die Ergebnisse automatisierter Tool-Reports mit dem relevanten Quellcode verknüpft und (c) auf dieser Grundlage priorisierte, entwicklernahe Handlungsempfehlungen erzeugt - lokal ausführbar und ohne den Transfer sensibler Anwendungsdaten an externe Dienste.

1.3 Zielsetzung und Forschungsfragen

Ziel dieser Bachelorarbeit ist die Konzeption und prototypische Umsetzung einer lokal ausführbaren, kontextsensitiven Assistenzlösung zur Barrierefreiheitsanalyse von Rich-Client-Webanwendungen. Als Untersuchungsobjekt dient eine React-basierte Webanwendung aus dem Umfeld der BITBW. Der Proof of Concept soll automatisierte Barrierefreiheits-Reports, Interaktionsdaten aus Playwright-Tests sowie ausgewählte Informationen aus der Codebasis in einer Datenpipeline zusammenführen. Auf dieser Grundlage werden priorisierte, entwicklernahe Handlungsempfehlungen abgeleitet und strukturiert aufbereitet. Die Arbeit liefert drei konkrete Beiträge: Erstens einen strukturierten Überblick darüber, wie unterschiedliche Datenquellen der Barrierefreiheitsanalyse systematisch kombiniert und interpretiert werden können. Zweitens einen Proof of Concept, der das Vorgehen exemplarisch demonstriert. Drittens eine qualitative Evaluation anhand definierter Kriterien, die aus den Anforderungen und der einschlägigen Fachliteratur abgeleitet werden.

Die übergeordnete Forschungsfrage dieser Arbeit lautet:

Inwieweit kann ein kontextsensitives KI-Assistenzsystem, das Tool-Reports, Playwright-Interaktionsdaten und Quellcode kombiniert, Barrierefreiheitsverstöße in React-basierten Webanwendungen automatisiert erkennen und entwicklernahe Handlungsempfehlungen generieren?

Diese übergeordnete Frage wird durch zwei konkretisierende Teilfragen (TF) operationalisiert:

- TF1: Welche Typen von Barrierefreiheitsverstößen - syntaktisch, semantisch und layoutbezogen - lassen sich durch den gewählten Ansatz zuverlässig erkennen, und wo liegen die methodischen Grenzen?
- TF2: Inwiefern verbessert die Einbeziehung von Quellcode und Playwright-Interaktionsdaten die Qualität der generierten Handlungsempfehlungen gegenüber rein DOM-basierten Ansätzen?

TF1 adressiert dabei das Erkennungsproblem und erlaubt eine differenzierte Aussage über die Leistungsfähigkeit und Grenzen des Systems entlang der Verstoß-Taxonomie von Fathallah et al.[24][25] TF2 greift den in der Literatur identifizierten Mehrwert der Quellcode-Kontextualisierung auf und überprüft ihn im Rahmen einer qualitativen Evaluation am gewählten Untersuchungsobjekt.

[24] Vgl. Fathallah, Hernández, Staab 2025
[25] Vgl. Šmite et al. 2012, S. 105-153

1.4 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich in neun Kapitel. Im Anschluss an die Einleitung werden in Kapitel 2 die theoretischen Grundlagen erarbeitet: Barrierefreiheit im Web, die WCAG-Standards und der rechtliche Rahmen, die Charakteristika von Rich-Client-Webanwendungen sowie Grundlagen zu Large Language Models und Prompt-Engineering-Strategien.

Kapitel 3 gibt einen systematischen Überblick über den Stand der Forschung. Die relevanten Arbeiten werden thematisch gegliedert - nach LLM-basierter Erkennung, Code-Korrektur, hybriden Pipelines und Evaluation - und in einer Konzeptmatrix nach Webster und Watson zusammengefasst.[26] Das Kapitel schließt mit einer expliziten Einordnung des Beitrags der vorliegenden Arbeit.

Kapitel 3 legt das methodische Vorgehen dar. Es beschreibt die Wahl der Fallstudienanalyse als wissenschaftliche Methode, definiert den Untersuchungsgegenstand und entwirft das Evaluationsdesign.[27] Kapitel 4 analysiert die verfügbaren Datenquellen - Tool-Reports, Playwright-Interaktionsdaten und Codebasis - und leitet daraus funktionale und nicht-funktionale Anforderungen ab.

Auf Basis dieser Anforderungen wird in Kapitel 6 das Assistenzsystem konzipiert: Systemarchitektur, Datenpipeline, Prompt-Strategie und Ausgabeformat werden begründet entworfen. Kapitel 6 beschreibt die prototypische Implementierung des Proof of Concept und illustriert die Systemfunktionalität anhand von Beispielabläufen für syntaktische, semantische und dynamisch erzeugte Barrierefreiheitsprobleme.

Kapitel 7 präsentiert und diskutiert die Evaluationsergebnisse, beantwortet die Forschungsfragen und reflektiert die Limitationen des Ansatzes. Das abschließende Kapitel 8 fasst die Kernaussagen zusammen, ordnet den Beitrag der Arbeit in die Forschungslandschaft ein und formuliert Empfehlungen für weiterführende Untersuchungen.

[26] Vgl. Webster, Watson 2002
[27] Vgl. Yin 2018

2 Theoretische Grundlagen

Das folgende Kapitel legt die Theoretischen Grundlagen für die Einführung des Proof of Concepts dar. Es werden Begriffe und Konzepte erläutert, die für das Verständnis der Problemstellung und des entwickelten Ansatzes erforderlich sind. Zunächst wird Barrierefreiheit in der Web als Untersuchungsgegenstand definiert (Abschnitt 2.1), anschließend der Stand automatisierter Barrierefreiheitstools sowie ihre Möglichkeiten und deren Grenzen (Abschnitt 2.2). Darauf aufabuend werden dann die besonderen Herausforderungen von Rich-Client-Anwendungen und das Werkzeug Playwright eingeführt (Abschnitt 2.3). Das Kapitel schließt mit einer Einführung in LLM, Modelle und relevante Prompting Strategien sowie auch Datenschutzbesorgungen (Abschnitt 2.4).

2.1 Barrierefreiheit von Webanwendungen

2.1.1 Begriffsdefinition und Relevanz

Der Begriff *Web Accessibility*, im Deutschen Web-Barrierefreiheit, bezeichnet nach der Definition der Web Accessibility Initivative (WAI) die Eigenschaft von Webangeboten, von allen Menschen unabhängig von körperlichen, motorischen oder sensorischen Einschränkungen sowie individuellen Fähigkeiten wahrgenommen werden zu können.[28] Web Accessibility geht jedoch über die reine Wahrnehmbarkeit hinaus und umfasst auch die Bedienbarkeit sowie Nutzbarkeit von Webangeboten.[29]. Laut Angaben der World Health Organization (WHO) wird geschätzt das rund 16% der Bevölkerung mit einer Form von Behinderung leben.[30] Wenn man aber noch temporäre Einschränkungen, wie gebrochene Hände oder situationsbedingte Einschränkungen wie gleißendes Sonnenlicht auf dem Bildschirm berücksichtigen möchte, würde die Zahl um weitere 1,5 Milliarden Menschen steigen.[31].Damit stellt Barrierefreiheit im Web nicht nur eine technische Herausforderung dar, sondern auch eine zentrale gesellschaftliche Aufgabe zur Gewährleistung digitaler Teilhabe. Das Recht auf barrierefreien Zugang zu Informations- und Kommunikationstechnologien ist seit 2006 in Art. 9 der UN-Behindertenrechtskonvention völkerrechtlich verankert.[32].

In der Forschungsliteratur werden vier wesentliche Behinderungsformen unterschieden, die jeweils spezifische Anforderungen an die Gestaltung barrierefreier Webangebote stellen.[33]

Visuelle Beeinträchtigungen umfassen vollständige oder partielle Blindheit sowie Farbfehlsichtigkeit.[34] Betroffene Personen nutzen häufig Screenreader-Software wie JAWS, NVDA oder Apple

[28] Vgl. Abuaddous, Jali, Basir 2016, S. 172
[29] Vgl. W3C 2025b
[30] Vgl. World Health Organization 2026
[31] Vgl. Organization 2026
[32] Vgl. United Nations 2006, S. 9
[33] Vgl. Lazar, Feng, Hochheiser 2017, S. 409-412
[34] Vgl. Gubbi Mohanbabu, Pavel 2024

VoiceOver, die die Webinhalte in eine verständliche Sprache umwandeln und dabei auf korrekte semantische HTML-Auszeichnung angewiesen ist.[35] Bilder werden oft als primäres Medium für Informationsaustausch genutzt und müssen somit auch korrekt von den Entwicklern oder Autoren beschrieben werden, damit Personen mit visuellen Einschränkungen nicht beeinträchtigt werden.[36] Ebenso ist eine vollständige Tastaturnavigation erforderlich, da viele Screenreader-Nutzer Webseiten primär über die Tastatur bedienen.

Auditive Beeinträchtigungen schließen Gehörlosigkeit und Schwerhörigkeit ein. Für diese Gruppe sind korrekte Untertitel und Transkriptionen von Audio- und Videoinhalten wesentlich. Darüber hinaus ist die Qualität entscheidend: Untertitel müssen synchron, vollsändig und leicht verständlich sein.[37] Für Live-Inhalte ist dies eine noch größere Herausforderung doch selbst hier haben sich in den letzten Jahren relevante Maßnahmen erwiesen wie Live-Captions oder Gebärdensprachedolmetschung. Die WCAG adressieren diese Anforderungen unter anderem in den Erfolgskriterien 1.2.x (zeitbasierte Medien) und 1.3.x/4.1.x (semantische Struktur und Kommunikation von Zuständen).[38][39]

Motorische Einschränkungen betreffen die Fähigkeit, konventionelle Eingabegeräte wie Maus oder Touchscreen präzise zu bedienen. Ursachen können beispielsweise neurologische Erkrankungen, Muskelschwäche oder eingeschränkte Feinmotorik sein, wodurch die Steuerung von Zeigegeräten erschwert wird.[40] Betroffene Personen sind daher häufig auf alternative Interaktionsformen angewiesen, etwa Tastaturnavigation, Sprachsteuerung oder spezielle assistive Eingabegeräte.[41] Eine vollständige Bedienbarkeit von Websites über die Tastatur ist deshalb eine zentrale Voraussetzung für barrierefreie Nutzung, da viele assistive Technologien auf Tastatureingaben aufbauen.[42] Fehlende Tastaturunterstützung oder komplexe Mausinteraktionen können daher erhebliche Barrieren darstellen.[43]

Kognitive und neurologische Beeinträchtigungen, darunter Lernschwächen, Dyslexie, Aufmerksamkeitsdefizit-Hyperaktivitätsstörung (ADHS) oder Demenz, können die Verarbeitung von Informationen, das Verständnis komplexer Inhalte sowie die Orientierung innerhalb von Webseiten erschweren.[44] Für betroffene Nutzer sind daher eine klare und verständliche Sprache, konsistente Strukturen sowie eine vorhersehbare Navigation besonders wichtig.[45] Darüber hinaus trägt eine reduzierte visuelle Komplexität, beispielsweise durch übersichtliche Layouts, ausreichend große Schrift und eine klare Informationshierarchie, dazu bei, kognitive Belastung zu verringern und die Nutzbarkeit von Webangeboten zu verbessern.[46]

[35] Vgl. Morris et al. 2016
[36] Vgl. World Wide Web Consortium (W3C) 2023, Erfolgskriterium 1.1.1
[37] Vgl. Harrenstien, Engineer 2026
[38] Vgl. World Wide Web Consortium (W3C) 2023
[39] Vgl. Abou-Zahra, Brewer, Cooper 2018, S. 2
[40] Vgl. Pérez et al. 2020, S. 1 ff.
[41] Vgl. Kouroupetroglou 2014
[42] Vgl. WebAIM 2026
[43] Vgl. Sarioğlu 2023, S 1 ff.
[44] Vgl. Lazar, Feng, Hochheiser 2017
[45] Vgl. World Wide Web Consortium (W3C) 2023
[46] Vgl. Abou-Zahra, Brewer, Cooper 2018

Assistive Technologien bilden die Voraussetzungen damit diese Nutzergruppen die digitale Inhalte und Angebote vollsändig ohne Einschränkungen nutzen können.[47] Zu den wichtigsten zählen Screenreader, Bildschirmvergrößerungssoftware, Eye-Tracking-Systeme sowie alternative Eingabegeräte.[48] Die Nützlichkeit dieser Technologien hängt stark von den Webanwendungen ab. So müssen diese klar definierte strukturelle und semantische Anfoderungen erfüllen.[49][50]. Die Nichterfüllung dieser Anforderungen führt zu einer Beeinträchtigung und Einschränkung für betroffene Personen und kann darüber hinaus wirtschaftliche Nachteile mit sich bringen. Studien zeigen, dass Unternehmen durch barrierefreie Webangebote ihre Reichweite signifikant erweitern und rechtliche Risiken durch Klagen nach dem BFSG oder der Datenschutz-Grundverordnung (DSGVO) vermeiden können.[51]

2.1.2 WCAG: Aufbau, Prinzipien und Konformitätsstufen

Die WCAG sind der internationale Standard für barrierefreie Webinhalte. Die Maßnahmen und Kriterien werden von dem World Wide Web Consortium auch bekannt als W3C im Rahmen der WAI herausgegeben. Die derzeit maßgebliche Referenzversion ist WCAG 2.1 aus dem Jahr 2018, die seit Oktober 2023 durch die WCAG 2.2 ergänzt wird.[52][53] Diese neuere Version ergänzt die ältere mit neun zusätzlichen eingeführten Erfolgskriterien und insbesondere Kognitive Einschränkungen und mobile Nutzung berücksichtigt.[54][55] Unter diesen Kriterien fallen konsistentere Fokusindikatoren (2.4.11-2.4.13) und die Vermeidung von Drag-and-Drop-Pflichtinteraktionen (2.5.7).[56]

Die WCAG ist in vier Schichten gegliedert: Prinzipien, Richtlinien, Erfolgskriterien und unterstützende Techniken. Auf der obersten Ebene stehen vier grundlegende Prinzipien, zusammengefast im Akronym **POUR**[57]:

- **Percievable (Wahrnehmbar):** Alle Informationen und Benutzerschnittstellen müssen für alle Nutzer und Nutzerinnen wahrnehmbar sein.[58] Typische Anforderungen sind Alternativtexte für Bilder (Erfolgskriterium 1.1.1) und ausreichende Farbkontraste (Erfolgskriterium 1.4.3).[59]

[47] Vgl. W3C 2025b
[48] Vgl. Abou-Zahra, Brewer, Cooper 2018, S. 2 f.
[49] Vgl. Morrison et al. 2017
[50] Vgl. Lazar, Feng, Hochheiser 2017, S. 415-420
[51] Vgl. Miesenberger, Peňáz, Kobayashi 2024, S. 1-9
[52] Vgl. World Wide Web Consortium (W3C) 2018
[53] Vgl. World Wide Web Consortium (W3C) 2023
[54] Vgl. Purwandari et al. 2025, S. 118
[55] Vgl. Souza et al. 2024
[56] Vgl. Purwandari et al. 2025, S. 118
[57] Vgl. Souza et al. 2024
[58] Vgl. World Wide Web Consortium (W3C) 2023, Abschnitt 1
[59] Vgl. Souza et al. 2024, S. 10

- **Operable (Bedienbar):** Alle Funktionen der Benutzeroberfläche müssen über die Tastatur erreichbar sein.[60] Zeitlich beschränkte Inhalte die nach einer vordefinierten Zeit verschwinden müssen auch verhindert werden. Auch Inhalte die Anfälle auslösen können müssen vermieden werden.[61]
- **Understandable (Verständlichkeit):** Inhalte und Bedienelemente müssen so verständlich gestaltet sein, dass die verwendete Sprache angegeben ist und Fehlermeldungen aussagekräftig sowie beschreibend formuliert sind.[62]
- **Robust (Robust):** Inhalte müssen so implementiert sein, dass sie von einer breiten Palette von aktuellen und zukünftigen assistiver Technologien interpretiert und ausgeführt werden können.[63]

Die 13 Richtlinien unterhalb der Prinzipien werden durch insgesamt 78 testbare Erfolgskriterien operationalisiert (WCAG 2.2).[64]. Jedes Erfolgskriterium ist einer von drei Konformitätstufen zugeordnet.[65] Stufe A umfasst grundlegende Anforderungen und dessen Nichteinhaltung bestimmte Nutzergruppen vollständig ausschließt. Die A stufigen Anforderungen sind auch bekannt als Mindestmindestanforderungen für Barrierefreiheit. Stufe AA ist der in Europa gesetzlich geforderte Mindeststandard für öffentliche Stellen und weitere Teile der Privatwirtschaft.[66] Stufe AAA beschreibt Anforderungen, die nicht für alle Inhalte generell erfüllbar sind. Die vorliegende Arbeit fokussiert entsprechend dem gesetzlichen Mindeststandard auf Stufe A und AA.

Die praktische Umsetzung dieser Standards ist trotz ihrer langen Existenz nach wie vor unzureichend.[67] Die schon in der Einleitung erwähnte WebAIM-Million-Studie 2025 belegt diese Begründung. Im Durschnitt wurden 56,8 WCAG-Fehler pro Seite identifiziert. Die häufigsten Verstoßkategorien betrafen fehlende Alternativtexte (54,5%), unzureichende Farbkontraste (80,6%), fehlende Formularetiketten (47,4%), leere Links (44,6%) und fehlende Sprachangaben im HTML (17,1%).[68]

2.1.3 Rechtlicher Rahmen

Auf europäischer Ebene bilder der European Accessibility Act (EAA) den übergeordneten rechtlichen Rahmen.[69] Er verpflichtet Mitgliedstaaten, einheitliche und Barrierefreikonforme Inhalte und Webangebote für die digitalen Produkten und Dienstleistungen durchzusetzen.

[60] Vgl. World Wide Web Consortium (W3C) 2023, Abschnitt 2
[61] Vgl. Souza et al. 2024
[62] Vgl. World Wide Web Consortium (W3C) 2023, Abschnitt 3
[63] Vgl. World Wide Web Consortium (W3C) 2023, Abschnitt 4
[64] Vgl. World Wide Web Consortium (W3C) 2023
[65] Vgl. Fathallah, Hernández, Staab 2025, S. 2
[66] Vgl. Kerkmann, Lewandowski 2015, S. 40, 195 ff.
[67] Vgl. Bassi et al. 2025
[68] Vgl. WebAIM 2025a
[69] Vgl. European Comission 2026

In Deutschland wurde der EAA durch das Barrierefreiheitsstärkungsgesetz (BFSG) in nationales Recht überführt.[70] Das BFSG ist seit dem 28. Juni 2025 für privatwirtschaftliche Anbierter verbindlich. Unternhemen mit weniger als zehn Beschäftigten und einem Jahresumsatz unter zwei Millionen Euro sind von bestimmten Pflichten ausgenommen.[71]

Für öffentliche Stellen des Bundes gilt in Deutschland bereits seit 2011 die Barrierefreie-Informationstechnik Verordnung (BITV) 2.0, die Behörden verpflichtet ihre Webangebote nach WCAG 2.1 auf mindestens Konformitätsstufe AA zu gestaltung und zu dokumentieren.[72] Die BITV 2.0 verweist in ihrer Anlage 1 unmittelbar auf die WCAG-Erfolgskriterien und ist somit dynamisch mit deren Weiterentwicklung verknüpft.

Für die IT Baden-Württemberg (BITBW) als zentrale IT-Dienstleister der Landesverwaltung Baden-Württembergs sind all diese rechtlichen Anforderungen unmittelbar handlungsrelevant.[73] Sämtliche von der BITBW entwickelten oder betriebenen Anwendungen müssen den Anforderungen entsprechen. Dies schafft eine exzellente Möglichkeit und begründet das praktische Interesse an effizienten, automatisierten Methoden der Barrierefreiheitsüberprüfung, wie sie in dieser Arbeit untersucht werden.

2.2 Automatisierte Barrierefreiheitsanalyse

Trotz der klar definierten Richtlinien der WCAG zeigt die Praxis, dass Barrierefreiheit in der Webentwicklung häufig unzureichend umgesetzt wird. Ein zentraler Grund dafür liegt in den Entwicklungsprozessen moderner Webanwendungen. In vielen Softwareprojekten wird Barrierefreiheit nicht von Beginn an als integraler Bestandteil der Entwicklung berücksichtigt, sondern erst in späten Testphasen oder gar nicht adressiert. Hinzu kommt, dass viele Entwicklerinnen und Entwickler nur begrenztes Wissen über die WCAG-Anforderungen besitzen und Accessibility-Aspekte nicht automatisch berücksichtigt werden.[74] Auch der zunehmende Einsatz komplexer Architekturen und Frameworks erschwert die korrekte Umsetzung barrierefreier Interaktionen. In Kombination mit Zeitdruck und fehlenden automatisierten Prüfverfahren führt dies dazu, dass Accessibility-Probleme häufig erst spät erkannt oder vollständig übersehen werden.[75]

2.2.1 Klassische Tool-Ansätze

Für die automatisierte Überprüfung von Webanwendungen auf Barrierefreiheit gibt es eine stetig wachsende Anzahl and Werkzeugen. Die WAI führt in ihrer aktuellen Evaluation-Tool-Liste über

[70] Vgl. Barrierefreiheitsstärkungsgesetz 2026
[71] Vgl. Bundesamt der Justiz und für Verbraucherschutz 2025, §1 (Anwendungsbereich) und §3 (Barrierefreiheit)
[72] Vgl. Bundesministerium für Digitales und Staatsmodernisierung 2025
[73] Vgl. BITBW 2021
[74] Vgl. Bassi et al. 2025, S. 298
[75] Vgl. Mowar 2024

160 Werkzeuge[76], von denen 84 explizit die WCAG 2.1 adressieren.[77] Alle gängigen Werkzeuge arbeiten regelbasiert, d. h. sie prüfen den DOM einer geladenen Seite anhand eines vordefinierten Regelkatalogs gegen bekannte Verstoßmuster. Das Document Object Model ist eine vom Browser erzeugte, baumartige Objektstruktur, die den HTML-Code einer Webseite als hierarchische Struktur aus Elementknoten repräsentiert und über Programmierschnittstellen von Skripten und Analysewerkzeugen zugänglich macht.[78]

Zu den meist genutzen automatisierten Barrierefreiheitstolls gehören **axe-core** entwickelt von Deque Systems, die sich als Standardlösung für automatische Integration von Tests in Pipelines etabliert hat.[79] **WAVE** by WebAIM bietet visuelle Browser-Extensions, die Fehler direkt in der gerenderten Seitenansicht annotiert.[80] **Lighthouse**, das in den Chrome-DevTools integriert ist und mehrere Funktionen wie Accessibility Testing, SEO-Metriken und Performanceanalysen bietet.[81] **IBM Equal Access Checker** und **ARC Toolkit** (TPGi) runden das Spektrum der verbreiteten Werkzeuge ab.

Der typische Output solcher Werkzeuge ist ein strukturierter Violation Report, bei dem jedem erkannten Verstoß eine Bezeichnung, Beschreibung und Schweregrad zugewiesen wird (Beispielsweise bei axe-core: minor, moderate, serious, critical), sowie falls möglich auch ein HTML-Ausschnittes des betroffenen Elements ausgegeben.[82] Dieser standardisierte Output bildet in der vorliegenden Arbeit die Grundlage für die erste Stufe der Datenpipeline.

2.2.2 Grenzen bestehender Tools

Trotz ihrer weiten Verbreitung weisen regelbasierte Accessibility-Tools fundamentale Grenzen auf. Diese lassen sich in drei Kategorien einteilen: begrentzte Regelabdeckung, semantische Blindheit und fehlende Dynamik.

Die **Regelabdeckung** der Werkzeuge ist erheblich geringer als oftmals angenommen. Kodithuwakku und Wickramaarachchi ermittelten, dass es leistungsfähige untersuchte Tools lediglich 21% aller WCAG-Tests automatisiert prüfen kann.[83][84] Für Tastaturzugänglichkeit und auditive Barrieren lag die Erkennungsrate in ihrer Studie bei null Prozent. Ein weiteres großes Problem bei ihrer Studie waren die *False Positives* die eine manuelle Nachprüfung erfordern.[85]Bassi et al. kommen zu dem Ergebnis, dass das beste verfügbare Einzeltool (**ARC Toolkit**) nur 26% der WCAG-Tests abdeckt.[86] Die Kombination mehrer Tools in einem Ensemble verbessert die Abdeckung, beseitigt aber das strukturelle Problem jedoch nicht vollständig. Pool zeigt, dass selbst

[76]Vgl. W3C 2025a
[77]Vgl. Delnevo, Andruccioli, Mirri 2024
[78]Vgl. Huang et al. 2024, S. 4
[79]Vgl. Deque Systems 2026
[80]Vgl. WebAIM 2025b
[81]Vgl. Google 2025
[82]Vgl. Kodithuwakku, Wickramaarachchi 2025, S. 5-6
[83]Vgl. Kodithuwakku, Wickramaarachchi 2025, S. 6
[84]Vgl. Tabelle 1
[85]Vgl. Kodithuwakku, Wickramaarachchi 2025, S. 9
[86]Vgl. Bassi et al. 2025

ein Ensemble aus acht gleichzeitig eingesetzten Tools, darunter auch axe-core, WAVE, Qualweb usw., erhebliche Lücken hinterlässt.[87] Insgesamt stellte sich in der Studie heraus das jedes einzelne Tool mindestens sieben Violations exklusiv erkennt, die von keinem anderem Tool gefunden werden. Problematisch ist dabei auch auch, dass die meisten Tools ihre eigene Abdeckung nicht kommunizieren: Manca et al. stellen fest, dass von elf untersuchten Werkzeugen lediglich drei explizit auf ihre Erkennungslücken hinweisen.[88]

Tool	**True Positives**	**False Positives**	**CER (%)**	**ICR (%)**
Lighthouse	17	16	52.94	15.92
WAVE	16	12	35.29	14.65
SiteImprove	23	18	47.06	19.75
axe	15	10	52.94	15.29
Accessibility Insights	24	19	47.06	15.29
A11y	6	8	23.53	3.82
Includia Accessibility Checker	33	31	41.18	21.02

Tab. 1: Vergleich der Erkennungsleistung verschiedener Accessibility-Prüfwerkzeuge (basierend auf Kodithuwakku und Wickramaarachchi). CER beschreibt den Anteil korrekt erkannter Verstöße im Verhältnis zu allen gemeldeten Verstößen, während ICR den Anteil tatsächlich vorhandener Accessibility-Probleme misst, die vom jeweiligen Tool erkannt wurden.

Das gravierendste Defizit liegt in der **semantischen Blindheit** der Werkzeuge. Regelbasierte Tools können lediglich prüfen, ob ein Accessibility-Attribut vorhanden ist, nicht aber, ob der Inhalt dem richtigem Zweck dient. Ein Bild mit dem Alternativtext *image* oder *IMG_0042.jpg* wird von allen Tools als regelkonform eingestuft, obwohl dieser Text für Screenreadernutzer und Nutzerrinnen keinen Mehrwert schafft. **Ma11y**, ein Mutation-Testing-Framework für Accessibility, hat dieses Problem empirisch quantifiziert: Bei der systematischen Injektion von 25 gezielten Defekten in reale Webanwendungen erkannten die sechs ausgewählten Tools semantische Verstöße nur zu 26%. Die Quote der erkannten syntaktischen Verstöße dagegen lag bei 54% .[89] Im Durschnitt blieben also nahezu die Hälfte aller injizierten Defekte untendeckt.[90]

Ein weiteres strukturelles Problem ist die **fehlende Dynamik** der Analyse. Viele Tools analysieren ausschließlich den initial geladenen DOM-Zustand einer Seite.[91] Barrieren die erst durch Nutzerinteraktionen enstehen, wie ein Hilfs- oder Modalfenster das geöffnet wird, ein Dropdownmenu um eine Kategorie auszuwählen oder eine Fehlermeldung erschient, bleiben dabei unsichtbar. Hinzu kommt dass alle Violation Reports generischer Natur sind. Sie beschreiben das Problem auf der Ebene des gerenderten DOM, ohne Bezug zur Quellcodestruktur der geprüften Anwendung herzustellen.[92]

[87] Vgl. Pool 2023
[88] Vgl. Manca et al. 2023, S. 4
[89] Vgl. Tafreshipour et al. 2024, S. 8-9
[90] Vgl. Tafreshipour et al. 2024
[91] Vgl. Fathallah, Hernández, Staab 2025, S. 5-7
[92] Vgl. Delnevo, Andruccioli, Mirri 2024, S. 5

2.2.3 Taxonomie der Verstöße: Syntaktisch, Semantisch, Layout

Um Barrierefreiheitsverstöße systematisch analysieren und gezielt adressieren zu lönnen wird in dieser Arbeit eine dreiteilige Taxonomie zugrunde gelegt die auf Fathallah et al. zurückgeht.[93]

- **Syntaktische Verstöße** bezeichnen Fälle, in denen ein erforderliches Accessibility-Element vollständig fehlt oder formal falsch gebildet ist.[94] Typische Beispiele sind ein fehlendes `alt`-Attribut an einem `<img>`-Element, ein fehlender `aria-label` an einer Schaltfläche ohne sichtbaren Text oder `<table>` ohne `<th>`-Elemente. Diese Kategorie ist für automatisierte regelbasierte Tools am besten prüfbar, da die Verletzung rein struktureller Natur ist.[95]
- **Semantische Verstöße** liegen vor, wenn Accessibility-Attributa zwar vorhanden sind, ihren Zweck aber inhaltlich verfehlen.[96] Ein Bild mit dem Alternativtext *Bild*, ein Formularelement mit der Beschriftung *Eingabe* oder ein Link mit dem Text *hier cklicken* sind typische Beispiele. Da die Bewertung solcher Verstöße Sprachverständnis und Kontexwissen erfordert, sind diese viel schwieriger von Tools zu entdecken und bilden damit das primäre Anwendungsfeld für LLM-basierte Analysen.[97]
- **Layout-Verstöße** betreffen visuelle Barrieren. Typische Probleme oder Barrieren sind unzureichende Farbkontraste[98], fehlende oder schwer erkennbare Fokusindikatoren[99] sowie eingeschränkte Zoomfähigkeit zählen zu dieser Kategorie. Einige dieser Verstöße können algorithmisch geprüft werden; andere erfordern ein kontextuelles visuelles Urteil.

2.3 Rich-Client-Webanwendungen und Testautomatisierung

2.3.1 Charakteristika und Abgrenzung

Der Begriff **Rich-Client-Webanwendung** bezeichnet Webanwendungen, bei denen ein Großteil der Anwendungslogik und Darstellungssteuerung clientseitig im Browser ausgeführt wird.[100] Im Gegensatz zu klassischen server-seitig gerenderten Anwendungen wird bei einer Single-Page Application (SPA) nach dem initialen Laden des HTML-Rahmens ausschließlich Daten zwischen Client und Server ausgetauscht.[101] Die Darstellung wird durch clientseitige Skriptsprachen wie JavaScript oder TypeScript direkt im Browser durch Manipulation des DOM gesteuert. React, das von Meta entwickelte JavaScript-Framework, ist der dominante Ansatz für die Entwicklung

[93] Vgl. Fathallah, Hernández, Staab 2025, S. 3 ff.
[94] Vgl. Tafreshipour et al. 2024, S. 3 f.
[95] Vgl. Flanagan 2020, S. 569 ff.
[96] Vgl. World Wide Web Consortium (W3C) 2023
[97] Vgl. He, Huq, Malek 2025
[98] Vgl. World Wide Web Consortium (W3C) 2023, Erfolgskriterium 1.4.3
[99] Vgl. World Wide Web Consortium (W3C) 2023, Erfolgskriterium 2.4.7-2.4.13
[100] Vgl. Meta Open Source o.J.a
[101] Vgl. Meta Open Source o.J.b

solcher Anwendungen. Laut Stack Overflow Developer Survey 2024 wird React von 44,7% aller befragten Entwicklerinnen und Entwickler eingesetzt.[102][103]

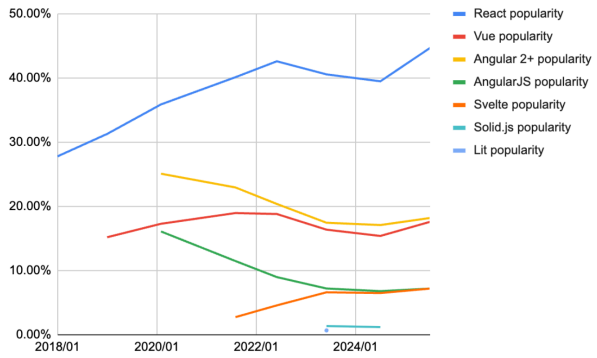

Abb. 1: Framework Popularity über die letzten Jahre[104]

React führt das Konzept eines virtuellen DOM ein: Statt direkte DOM-Manipulationen vorzunehmen, erzeugt React zunächst eine speicherinterne Darstellung des Ziel-DOM-Zustands, vergleicht diese mit dem aktuellen DOM (Reconciliation) und führt nur die notwendigen minimalen Änderungen durch: „The virtual DOM is a programming concept where an ideal, or 'virtual', representation of a UI is kept in memory and synced with the 'real' DOM".[105] Anwendungslogik und Darstellung werden in wiederverwendbaren **Komponenten** gekapselt. Die Kommunikation zwischen Komponenten erfolgt über **Props** (unveränderliche Eingabeparameter) sowie über den internen **State** (veränderlicher Zustand). Bei der Verwendung von TypeScript werden die Strukturen dieser Daten durch statische Typdefinitionen beschrieben, wodurch eine frühzeitige Fehlererkennung und bessere Wartbarkeit des Codes ermöglicht wird. Die Darstellung der Komponenten wird mithilfe der JavaScript-Syntaxerweiterung JSX definiert.[106]

2.3.2 Herausforderungen für die Barrierefreiheitsprüfung

Die Charakteristika von React-Anwendungen schaffen mehrere Herausforderungen für die Barrierefreiheitsanalyse, die über die allgemeinen Grenzen regelbasierter Tools hinausgehen.[107]

Erstens entstehen viele Barrierefreiheitsprobleme **zustandsabhängig**, wie schon in Kapitel 2.2.2 erläutert. Ein Modalfenster, das nach einem Klick geöffnet wird und den Tastaturfokus nicht korrekt verwaltet;[108]eine Fehlermeldung, die nach dem Absenden eines Formulars erscheint und nicht durch eine ARIA-Live-Region programmatisch angekündigt wird; ein Dropdown-Menü, das

[102]Vgl. Stack Overflow 2025
[103]Hier den Anhang noch zitieren mit mehr Grafiken und die Footnote von dem Bild korrigieren/richtig einfügen
[105]Etnommen aus Meta Open Source o.J.c
[106]Vgl. Bai 2022
[107]Vgl. Tafreshipour et al. 2024, S. 102 f.
[108]Vgl. He, Huq, Malek 2025, S. 5-6

für Tastaturnutzer nicht navigierbar ist - all diese Probleme sind im initialen Seitenzustand nicht vorhanden und damit für statische Analysewerkzeuge unsichtbar.[109] [110]

Zweitens besteht eine **Diskrepanz zwischen Quellcode und gerendertem DOM**. In React werden Benutzeroberflächen in JSX definiert; der im Browser gerenderte DOM ist das Ergebnis der React-Laufzeitumgebung, die JSX-Komponenten in DOM-Elemente übersetzt und State-Änderungen koordiniert. Ein Accessibility-Tool, das den gerenderten DOM analysiert, sieht zwar das Ergebnis dieses Prozesses aber nicht aber den Quellcode, aus dem es hervorging.[111] Für Entwicklerinnen und Entwickler ist jedoch der Quellcode der relevante Interventionspunkt.

Drittens erschwert die Komponentenabstraktion die Ursachenanalyse: Ein einzelnes, in vielen Kontexten eingesetztes Komponenten-Element kann ein Accessibility-Problem aufweisen, das sich im gerenderten DOM an zahlreichen Stellen manifestiert. Accessibility-Tool-Reports zeigen jeden einzelnen Treffer - ohne den Bezug zur Quellkomponente herzustellen, deren einmalige Korrektur alle Treffer beseitigen würde.[112]

2.3.3 Playwright als Testautomatisierungswerkzeug

Um diese dynamischen Zustände automatisiert reproduzierbar testen zu können, werden Werkzeuge zur Browserautomatisierung benötigt.

Playwright ist ein von Microsoft entwickeltes Open-Source-Framework für die End-to-End-Testautomatisierung von Webanwendungen.[113] Es ermöglicht die automatisierte Steuerung der Browser Chromium, Firefox und WebKit über eine einheitliche API und führt Anwendungen in einem echten Browser-Kontext aus, sodass JavaScript-Ausführung, clientseitiges Rendering und dynamische DOM-Manipulationen vollständig nachgebildet werden.

Für die Barrierefreiheitsanalyse ist Playwright aus mehreren Gründen besonders geeignet. Erstens kann es durch programmierte Interaktionen - Klicks, Formularausfüllungen, Tastatureingaben, Hover-Aktionen - gezielt verschiedene Anwendungszustände auslösen. Zweitens bietet Playwright mit `@axe-core/playwright` eine direkte Integration der axe-core-Bibliothek: Nach jeder Interaktion kann unmittelbar eine vollständige axe-Prüfung des aktuellen DOM-Zustands angestoßen werden.[114]

Huang et al. nutzen in **ACCESS** genau diese Integration für die sequentielle Prüfung verschiedener Seitenzustände.[115] Fathallah et al. zeigen in **AccessGuru**, dass die Kombination aus Playwright-Interaktion und axe-core-Analyse entscheidend dazu beiträgt, auch dynamisch

[109] Vgl. World Wide Web Consortium (W3C) 2023, Erfolgskriterium 2.4.3 & 4.1.3, jeweils Stufe A
[110] Vgl. Kapitel 2.2.2
[111] Vgl. Fernandes et al. 2012
[112] Vgl. Abou-Zahra, Brewer, Cooper 2018
[113] Vgl. Microsoft 2026
[114] Vgl. Deque Systems 2026
[115] Vgl. Huang et al. 2024

erzeugte Barrieren zu erfassen.[116] He et al. nutzen Playwright in **GenA11y** für die DOM-Extraktion nach vollständigem clientseitigem Rendering.[117] In der vorliegenden Arbeit dient Playwright als zentrales Werkzeug der Detection-Phase: Es löst definierte Interaktionssequenzen aus, erfasst die resultierenden DOM-Zustände und stößt axe-core-Prüfungen an, deren strukturierter JSON-Output als erster Input für die LLM-Analysepipeline dient. Dieser Ansatz bildet die Grundlage für die Detection-Phase des in dieser Arbeit entwickelten Systems zur automatisierten Barrierefreiheitsanalyse.

2.4 KI-Assistenzsysteme für Softwareanalyse und Handlungsempfehlungen

[116] Vgl. Fathallah, Hernández, Staab 2025, S. 2 f.
[117] Vgl. He, Huq, Malek 2025

3 Methodik

3.1 Wissenschaftliche Methode Fallstudienanalyse

Studie		Konzepte												
		Ziel			Datenquellen			KI-/LLM-Ansatz			Evaluation			Ko
Autor(en) & Jahr	**System**	WCAG-Konf. prüfen	Barrieren korrigieren	Entwickler unterstützen	Tool-Reports (axe etc.)	Interaktionsdaten (Playwright)	Quellcode-Analyse	LLM-basiert	Kontextsensitiv / RAG	Lokal ausführbar	Automatisierte Eval.	Qualitative Eval.	Nutzerstudie	Rich-Client / SPA
Hoch priorisierte Studien														
He et al. (2025)	GenA11y	x			x			x			x			
Fathallah et al. (2025)	AccessGuru	x	x	x	x	x		x			x			
Bassi et al. (2025)	LLM Auditing	x	x	x				x				x		
Paternò et al. (2025)	TeVal	x		x	x			x	x			x		
Tafreshipour et al. (2024)	Ma11y	x			x	x					x			
Martinez Campo (2026)	**ACE**	x	x	x	x	x	x	x	x	x		x		x

Tab. 2: Konzeptmatrix nach Webster Watson[118]: Hoch priorisierte Studien im Vergleich zum eigenen Proof of Concept. **x** = Konzept vorhanden/adressiert; leer = nicht adressiert. Die vollständige Matrix aller analysierten Studien findet sich in Anhang 1.

3.2 Untersuchungsgegenstand und Scope-Abgrenzung

3.3 Evaluationsdesign und Kriterien

4 Analyse und Anforderungen

4.1 Analyse der Tool-Outputs und typischer Findings

4.2 Analyse von Interaktionsdaten aus Playwright

4.3 Analyse der Codebasis als Kontextquelle

4.4 Datenschutz & Sicherheitsanforderungen

5 Konzeption des Assistenzsystems

5.1 Zielarchitektur und Datenpipeline

5.2 Modell der Kontextualisierung & Priorisierung

5.3 Ausgabeformat entwicklernahe To-Do-Liste

6 Implementierung des Proof of Concept

6.1 Technische Umsetzung

6.2 Beispielabläufe und Ergebnisse am Untersuchungsobjekt

7 Evaluation & Diskussion

7.1 Ergebnisse entlang der Kriterien und Limitationen

7.2 Handlungsempfehlungen und Ausblick

8 Fazit

Anhang

Anhangverzeichnis

Anhang 1 Erweiterte Konzeptmatrix 24

Anhang 1: Erweiterte Konzeptmatrix

Studie		**Konzepte**												
		Ziel			**Datenquellen**			**KI-/LLM-Ansatz**			**Evaluation**			**Ko**
Autor(en) & Jahr	**System**	WCAG-Konf. prüfen	Barrieren korrigieren	Entwickler unterstützen	Tool-Reports (axe etc.)	Interaktionsdaten (Playwright)	Quellcode-Analyse	LLM-basiert	Kontextsensitiv / RAG	Lokal ausführbar	Automatisierte Eval.	Qualitative Eval.	Nutzerstudie	Rich-Client / SPA
Hoch priorisierte Studien														
He et al. (2025)	GenA11y	x			x			x			x			
Fathallah et al. (2025)	AccessGuru	x	x	x	x	x		x			x			
Bassi et al. (2025)	LLM Auditing	x	x	x				x				x		
Paternò et al. (2025)	TeVal	x		x	x			x	x			x		
Tafreshipour et al. (2024)	Ma11y	x			x	x					x			
Fernández-Navarro & Chicano (2025)	LLM Remediation	x	x	x	x		x	x			x			x
Mittel priorisierte Studien														
Yu et al. (2025)	GenAI Screen Reader	x	x		x		x	x	x		x	x	x	
Huang et al. (2024)	ACCESS	x	x	x	x	x		x			x			
Pool (2023)	Testaro	x			x	x					x			
Mowar et al. (2024)	Copilot Study	x		x				x					x	
Abu Doush & Kassem (2024)	AI Code Study	x	x	x				x			x			
Guriță & Vatavu (2025)	AI UI Study	x						x			x			
Kodithuwakku & Wick. (2025)	Tool Eval.	x			x						x			
Manca et al. (2023)	Transparency	x		x	x							x	x	
Jiang & Nam (2025)	Cursor Rules			x			x	x				x		
Gubbi Mohanbabu & Pavel (2024)	Context-Aware Img.	x	x					x	x		x	x	x	
Campoverde-Molina & Luján-Mora (2025)	AI A11y SMS	x	x	x	x			x				x		
Leedy et al. (2025)	GenAI Builder Eval.	x			x			x			x	x		
Patel & Melcer (2025)	AIDA Study	x		x				x				x	x	
Aljedaani et al. (2024)	ChatGPT A11y Code	x	x	x			x	x			x			
Niedrig priorisierte Studien														
Abou-Zahra et al. (2018)	AI for A11y	x						x				x		
Delnevo et al. (2024)	LLM Interaction	x	x					x				x		x
Draffan et al. (2020)	Web2Access+AI	x			x			x				x		
Eigener Proof of Concept														
Martinez Campo (2025)	**ACE**	x	x	x	x	x	x	x	x	x		x		x

Tab. 3: Konzeptmatrix nach Webster Watson[119]: Vollständige Übersicht aller analysierten Studien im Vergleich zum eigenen Proof of Concept. **x** = Konzept vorhanden/adressiert; leer = nicht adressiert.

Literaturverzeichnis

Abou-Zahra, Shadi; Brewer, Judy; Cooper, Michael (2018): Artificial Intelligence (AI) for Web Accessibility: Is Conformance Evaluation a Way Forward? In: *Proceedings of the 15th International Web for All Conference*. W4A '18. New York, NY, USA: Association for Computing Machinery, S. 1–4. ISBN: 978-1-4503-5651-0. DOI: 10.1145/3192714.3192834. URL: https://dl.acm.org/doi/10.1145/3192714.3192834 (Abruf: 24. 02. 2026).

Abuaddous, Hayfa Y.; Jali, Mohd Zalisham; Basir, Nurlida (2016): Web Accessibility Challenges. In: *International Journal of Advanced Computer Science and Applications (ijacsa)* 7.10. ISSN: 2156-5570. DOI: 10.14569/IJACSA.2016.071023. URL: https://thesai.org/Publications/ViewPaper?Volume=7&Issue=10&Code=ijacsa&SerialNo=23 (Abruf: 04. 03. 2026).

Bai, Aiden (2022): A Fast Compiler-Augmented Virtual DOM for the Web. In: *arXiv*. URL: https://arxiv.org/abs/2202.08409.

Barrierefreiheitsstärkungsgesetz (2026): BFSG (Barrierefreiheitsstärkungsgesetz). URL: https://bfsg-gesetz.de/ (Abruf: 24. 02. 2026).

Bassi, Barry; Delnevo, Giovanni; Franco, Mirko; Gaggi, Ombretta; Gatto, Salvatore; Mirri, Silvia; Olaiya, Kelvin (2025): An Assessment of LLM-Based Auditing and Validation for Web Accessibility. In: *Proceedings of the 2025 International Conference on Information Technology for Social Good*. GoodIT '25. New York, NY, USA: Association for Computing Machinery, S. 297–305. ISBN: 979-8-4007-2089-5. DOI: 10.1145/3748699.3749805. URL: https://dl.acm.org/doi/10.1145/3748699.3749805 (Abruf: 24. 02. 2026).

BITBW (2021): 6 Jahre BITBW. URL: https://bitbw.de/neuigkeiten/artikel/6-jahre-bitbw-1 (Abruf: 12. 07. 2025).

Bundesamt der Justiz und für Verbraucherschutz (2025): Gesetz zur Umsetzung der Richtlinie (EU) 2019/882 des Europäischen Parlaments und des Rates über die Barrierefreiheitsanforderungen für Produkte und Dienstleistungen 1. URL: https://www.gesetze-im-internet.de/bfsg/ (Abruf: 12. 07. 2025).

Bundesministerium für Digitales und Staatsmodernisierung (2025): Barrierefreie-Informationstec Verordnung (BITV 2.0). URL: https://www.barrierefreiheit-dienstekonsolidierung.bund.de/Webs/PB/DE/gesetze-und-richtlinien/bitv2-0/bitv2-0-node.html (Abruf: 24. 02. 2026).

Delnevo, Giovanni; Andruccioli, Manuel; Mirri, Silvia (2024): On the Interaction with Large Language Models for Web Accessibility: Implications and Challenges. In: *2024 IEEE 21st Consumer Communications & Networking Conference (CCNC)*. 2024 IEEE 21st Consumer Communications & Networking Conference (CCNC). ISSN: 2331-9860, S. 1–6. DOI: 10.1109/CCNC51664.2024.10454680. URL: https://ieeexplore.ieee.org/document/10454680 (Abruf: 24. 02. 2026).

Deque Systems (2026): *dequelabs/axe-core*. original-date: 2015-06-10T15:26:45Z. URL: https://github.com/dequelabs/axe-core (Abruf: 24. 02. 2026).

European Comission (2026): European accessibility act (EAA). URL: https://commission.europa.eu/strategy-and-policy/policies/justice-and-fundamental-rights/disability/european-accessibility-act-eaa_en (Abruf: 2026).

Fathallah, Nadeen; Hernández, Daniel; Staab, Steffen (2025): AccessGuru: Leveraging LLMs to Detect and Correct Web Accessibility Violations in HTML Code. In: *Proceedings of the 27th International ACM SIGACCESS Conference on Computers and Accessibility*. ASSETS '25. New York, NY, USA: Association for Computing Machinery, S. 1–22. ISBN: 979-8-4007-0676-9. DOI: 10.1145/3663547.3746360. URL: https://dl.acm.org/doi/10.1145/3663547.3746360 (Abruf: 24. 02. 2026).

Fernandes, Nádia; Costa, Daniel; Duarte, Carlos; Carriço, Luís (2012): Evaluating the Accessibility of Web Applications. In: *Procedia Computer Science*. Proceedings of the 4th International Conference on Software Development for Enhancing Accessibility and Fighting Info-exclusion (DSAI 2012) 14, S. 28–35. ISSN: 1877-0509. DOI: 10.1016/j.procs.2012.10.004. URL: https://www.sciencedirect.com/science/article/pii/S1877050912007661 (Abruf: 05. 03. 2026).

Flanagan, David (2020): JavaScript: The definitive guide. O'Reilly. ISBN: 0596805527.

Google (2025): Lighthouse : Chrome for developers. URL: https://developer.chrome.com/docs/lighthouse/overview (Abruf: 06. 10. 2025).

Gubbi Mohanbabu, Ananya; Pavel, Amy (2024): Context-Aware Image Descriptions for Web Accessibility. In: *Proceedings of the 26th International ACM SIGACCESS Conference on Computers and Accessibility*. ASSETS '24. New York, NY, USA: Association for Computing Machinery, S. 1–17. ISBN: 979-8-4007-0677-6. DOI: 10.1145/3663548.3675658. URL: https://dl.acm.org/doi/10.1145/3663548.3675658 (Abruf: 24. 02. 2026).

Harrenstien, Ken; Engineer, Software (2026): Automatic captions in YouTube. Official Google Blog. URL: https://googleblog.blogspot.com/2009/11/automatic-captions-in-youtube.html (Abruf: 04. 03. 2026).

He, Ziyao; Huq, Syed Fatiul; Malek, Sam (2025): Enhancing Web Accessibility: Automated Detection of Issues with Generative AI. In: *Proc. ACM Softw. Eng.* 2 (FSE), FSE101:2264–FSE101:2287. DOI: 10.1145/3729371. URL: https://dl.acm.org/doi/10.1145/3729371 (Abruf: 24. 02. 2026).

Heinemann, Daniela (2024): „Barrierefreiheit". In: *Digitalisierte Verwaltung - Vernetztes E-Government*. Hrsg. von Margrit Seckelmann. Berlin: Erich Schmidt Verlag GmbH & Co. KG, S. 469–488. ISBN: 978-3-503-23763-0. DOI: 10.37307/b.978-3-503-23763-0.15. URL: https://doi.org/10.37307/b.978-3-503-23763-0.15 (Abruf: 24. 02. 2026).

Huang, Calista; Ma, Alyssa; Vyasamudri, Suchir; Puype, Eugenie; Kamal, Sayem; Garcia, Juan Belza; Cheema, Salar; Lutz, Michael (2024): ACCESS: Prompt Engineering for Automated Web Accessibility Violation Corrections. DOI: 10.48550/arXiv.2401.16450. arXiv: 2401.16450[cs]. URL: http://arxiv.org/abs/2401.16450 (Abruf: 25. 02. 2026).

Kerkmann, Friederike; Lewandowski, Dirk (2015): Barrierefreie Informationssysteme: Zugänglichkeit für Menschen mit Behinderung in Theorie und Praxis. Google-Books-ID: T5ylCQAAQBAJ. Walter de Gruyter GmbH & Co KG. 292 S. ISBN: 978-3-11-033729-7.

Kodithuwakku, Tashmi; Wickramaarachchi, Dilani (2025): Assessing the Limits of Automated Accessibility Testing: Insights for QA Engineers and Tool Developers. In: *2025 5th International Conference on Advanced Research in Computing (ICARC)*. 2025 5th International Conference on Advanced Research in Computing (ICARC), S. 1–6. DOI: 10.1109/ICARC64760.2025.10963173. URL: https://ieeexplore.ieee.org/document/10963173 (Abruf: 24. 02. 2026).

Kouroupetroglou, Georgios (2014): Assistive Technologies and Computer Access for Motor Disabilities. Publication Title: https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/978-1-4666-4438-0. IGI Global Scientific Publishing. ISBN: 978-1-4666-4438-0. URL: https://www.igi-global.com/book/assistive-technologies-computer-access-motor/www.igi-global.com/book/assistive-technologies-computer-access-motor/75832 (Abruf: 04. 03. 2026).

Krotoff, Tanguy (2026): Front-end frameworks popularity (React, Vue, Angular and Svelte). Gist. URL: https://gist.github.com/tkrotoff/b1caa4c3a185629299ec234d2314e190 (Abruf: 05. 03. 2026).

Landesrecht BW (2015): Gesetz zur Errichtung der Landesoberbehörde BITBW. URL: https://www.landesrecht-bw.de/bsbw/document/jlr-ITErGBWV3P2 (Abruf: 12. 07. 2025).

Lazar, Jonathan; Feng, Jinjuan Heidi; Hochheiser, Harry (2017): Research Methods in Human-Computer Interaction. Elsevier. ISBN: 978-0-12-805390-4.

Manca, Marco; Palumbo, Vanessa; Paternò, Fabio; Santoro, Carmen (2023): The Transparency of Automatic Web Accessibility Evaluation Tools: Design Criteria, State of the Art, and User Perception. In: *ACM Trans. Access. Comput.* 16.1, 3:1–3:36. ISSN: 1936-7228. DOI: 10.1145/3556979. URL: https://dl.acm.org/doi/10.1145/3556979 (Abruf: 24. 02. 2026).

Meta Open Source (o.J.a). URL: https://react.dev/.

Meta Open Source (o.J.b): Render and Commit. URL: https://react.dev/learn/render-and-commit.

Meta Open Source (o.J.c): Virtual DOM and Internals. URL: https://legacy.reactjs.org/docs/faq-internals.html.

Microsoft (2026): Installation | Playwright. URL: https://playwright.dev/docs/intro (Abruf: 24. 02. 2026).

Miesenberger, Klaus; Peňáz, Petr; Kobayashi, Makoto, Hrsg. **(2024)**: *Computers Helping People with Special Needs: 19th International Conference, ICCHP 2024, Linz, Austria, July 8–12, 2024, Proceedings, Part I*. Bd. 14750. Lecture Notes in Computer Science. Cham: Springer Nature Switzerland. ISBN: 978-3-031-62845-0 978-3-031-62846-7. DOI: 10.1007/978-3-031-62846-7. URL: https://link.springer.com/10.1007/978-3-031-62846-7 (Abruf: 04. 03. 2026).

Morris, Meredith Ringel; Zolyomi, Annuska; Yao, Catherine; Bahram, Sina; Bigham, Jeffrey P.; Kane, Shaun K. (2016): "With most of it being pictures now, I rarely use it: Understanding Twitter's Evolving Accessibility to Blind Users. In: *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*. CHI '16. New York, NY, USA: Association for Computing Machinery, S. 5506–5516. ISBN: 978-1-4503-3362-7. DOI: 10.1145/2858036.2858116. URL: https://dl.acm.org/doi/10.1145/2858036.2858116 (Abruf: 04. 03. 2026).

Morrison, Cecily; Cutrell, Edward; Dhareshwar, Anupama; Doherty, Kevin; Thieme, Anja; Taylor, Alex (2017): Imagining Artificial Intelligence Applications with People with Visual Disabilities using Tactile Ideation. In: *Proceedings of the 19th International ACM SIGACCESS Conference on Computers and Accessibility*. ASSETS '17. New York, NY, USA: Association for Computing Machinery, S. 81–90. ISBN: 978-1-4503-4926-0. DOI: 10.1145/3132525.3132530. URL: https://dl.acm.org/doi/10.1145/3132525.3132530 (Abruf: 04. 03. 2026).

Mowar, Peya (2024): Accessibility in AI-Assisted Web Development. In: *Proceedings of the 21st International Web for All Conference*. W4A '24. New York, NY, USA: Association for Computing Machinery, S. 123–125. ISBN: 979-8-4007-1030-8. DOI: 10.1145/3677846.3679054. URL: https://dl.acm.org/doi/10.1145/3677846.3679054 (Abruf: 24. 02. 2026).

Organization, World Health (2026): World Report on Disability. URL: https://www.who.int/teams/noncommunicable-diseases/sensory-functions-disability-and-rehabilitation/world-report-on-disability (Abruf: 04. 03. 2026).

Pérez, J. Eduardo; Arrue, Myriam; Valencia, Xabier; Abascal, Julio (2020): Longitudinal Study of Two Virtual Cursors for People With Motor Impairments: A Performance and Satisfaction Analysis on Web Navigation. In: *IEEE Access*. DOI: 10.1109/ACCESS.2020.3001766.

Pool, Jonathan Robert (2023): Testaro: Efficient Ensemble Testing for Web Accessibility. In: *Proceedings of the 25th International ACM SIGACCESS Conference on Computers and Accessibility*. ASSETS '23. New York, NY, USA: Association for Computing Machinery, S. 1–4. ISBN: 979-8-4007-0220-4. DOI: 10.1145/3597638.3614505. URL: https://dl.acm.org/doi/10.1145/3597638.3614505 (Abruf: 24. 02. 2026).

Purwandari, Nuraini; Dewi, Ratna Kusuma; Rinaldo; Sucipto, Purwo Agus (2025): Policy in Practice: A Systematic Review of WCAG 2.2 and ADA 2024 Effects on Web and Mobile Accessibility. In: *ResearchGate*. DOI: 10.61978/digitus.v3i2.957. URL: https://www.researchgate.net/publication/396366120_Policy_in_Practice_A_Systematic_Review_of_WCAG_22_and_ADA_2024_Effects_on_Web_and_Mobile_Accessibility (Abruf: 04. 03. 2026).

Sarioğlu, Aylin (2023): Accessibility in Web Modeling Tools: Systematic Review and Conceptualization of a Keyboard-Only Prototype. In.

Šmite, Darja; Wohlin, Claes; Galviņa, Zane; Prikladnicki, Rafael (2012): An empirically based terminology and taxonomy for Global Software Engineering. In: *Empirical Software Engineering* 19.1, S. 105–153. DOI: 10.1007/s10664-012-9217-9.

Souza, Aline; de Souza Filho, José Cezar; Bezerra, Carla; Alves, Victor Anthony; Lima, Lara; Marques, Anna Beatriz; Teixeira Monteiro, Ingrid (2024): Accessibility Evaluation of Web Systems for People with Visual Impairments: Findings from a Literature Survey. In: *Proceedings of the XXIII Brazilian Symposium on Human Factors in Computing Systems*. IHC '24. New York, NY, USA: Association for Computing Machinery, S. 1–13. ISBN: 979-8-4007-1224-1. DOI: 10.1145/3702038.3702090. URL: https://dl.acm.org/doi/10.1145/3702038.3702090 (Abruf: 04. 03. 2026).

Stack Overflow (2025): Survey index. URL: https://survey.stackoverflow.co/2025/.

Tafreshipour, Mahan; Deshpande, Anmol; Mehralian, Forough; Ahmed, Iftekhar; Malek, Sam (2024): Ma11y: A Mutation Framework for Web Accessibility Testing. In: *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. ISSTA 2024. New York, NY, USA: Association for Computing Machinery, S. 100–111. ISBN: 979-8-4007-0612-7. DOI: 10.1145/3650212.3652113. URL: https://dl.acm.org/doi/10.1145/3650212.3652113 (Abruf: 24. 02. 2026).

United Nations (2006): Convention on the Rights of Persons with Disabilities. United Nations. URL: https://www.un.org/disabilities/documents/convention/convoptprot-e.pdf (Abruf: 04. 03. 2026).

United Nations Human Rights (2026). URL: https://www.ohchr.org/en/instruments-mechanisms/instruments/convention-rights-persons-disabilities (Abruf: 28. 02. 2026).

W3C (2025a): Evaluation-Tool-List. URL: https://www.w3.org/WAI/test-evaluate/tools/list/ (Abruf: 06. 10. 2025).

W3C (2025b): Web Accessibility Initivative. URL: https://www.w3.org/WAI/ (Abruf: 06. 10. 2025).

WebAIM (2019): The WebAIM Million - 2019 report on the accessibility of the top 1,000,000 home pages. URL: https://webaim.org/projects/million/2019 (Abruf: 24. 02. 2026).

WebAIM (2025a): The WebAIM Million - 2025 report on the accessibility of the top 1,000,000 home pages. URL: https://webaim.org/projects/million/ (Abruf: 24. 02. 2026).

WebAIM (2025b): Wave Web Accessibility Evaluation Tools. URL: https://wave.webaim.org/ (Abruf: 06. 10. 2025).

WebAIM (2026): WebAIM: Keyboard Accessibility. URL: https://webaim.org/techniques/keyboard/ (Abruf: 04. 03. 2026).

Webster, Jane; Watson, Richard T. (2002): Analyzing the Past to Prepare for the Future: Writing a Literature Review. In: *MIS Quarterly* 26.2, S. 2–6.

World Health Organization (2026). URL: https://www.who.int/health-topics/disability#tab=tab_1 (Abruf: 28. 02. 2026).

World Wide Web Consortium (W3C) (2018): Web Content Accessibility Guidelines (WCAG) 2.2. URL: https://www.w3.org/TR/WCAG21/ (Abruf: 24. 02. 2026).

World Wide Web Consortium (W3C) (2023): Web Content Accessibility Guidelines (WCAG) 2.2. URL: https://www.w3.org/TR/WCAG22/ (Abruf: 24. 02. 2026).

Yin, Dr Robert K. (2018): Case Study Research and Applications: Design and Methods. Los Angeles London New Delhi Singapore Washington DC Melbourne: SAGE Publications, Inc. 352 S. ISBN: 978-1-5063-3616-9.

Erklärung zur Verwendung generativer KI-Systeme

Bei der Erstellung der eingereichten Arbeit habe ich auf künstlicher Intelligenz (KI) basierte Systeme benutzt:

☒ ja

☐ nein[120]

Falls ja: Die nachfolgend aufgeführten auf künstlicher Intelligenz (KI) basierten Systeme habe ich bei der Erstellung der eingereichten Arbeit benutzt:

1.

2.

3. ...

Ich erkläre, dass ich

- mich aktiv über die Leistungsfähigkeit und Beschränkungen der oben genannten KI-Systeme informiert habe,[121]
- die aus den oben angegebenen KI-Systemen direkt oder sinngemäß übernommenen Passagen gekennzeichnet habe,
- überprüft habe, dass die mithilfe der oben genannten KI-Systeme generierten und von mir übernommenen Inhalte faktisch richtig sind,
- mir bewusst bin, dass ich als Autorin bzw. Autor dieser Arbeit die Verantwortung für die in ihr gemachten Angaben und Aussagen trage.

Die oben genannten KI-Systeme habe ich wie im Folgenden dargestellt eingesetzt:

Arbeitsschritt in der wissenschaftlichen Arbeit	**Eingesetzte(s) KI-System(e)**	**Beschreibung der Verwendungsweise**

[120] Die Erklärung ist in jedem Fall zu unterzeichnen, auch wenn Sie keine KI-Systeme genutzt haben und Ihr Kreuz bei „nein“ gesetzt haben.

[121] U.a. gilt es hierbei zu beachten, dass an KI weitergegebene Inhalte ggf. als Trainingsdaten genutzt und wiederverwendet werden. Dies ist insb. für betriebliche Aspekte als kritisch einzustufen.

(Ort, Datum) (Unterschrift)

Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit mit dem Thema: *Kontextsensitive, lokale KI-Assistenz zur automatisierten Barrierefreiheitsanalyse von Rich-Client-Webanwendungen mittels Tool-Reports, Playwright-Interaktionen und Codeanalyse* selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

(Ort, Datum) (Unterschrift)